

```

@NotBlank
private String value;
Name() {}
...
}

```

As you can recollect, it follows the immutable *ValueObject* pattern. The annotation `@Embeddable` marks the properties of *Name* to be part of another table, which is *UMS_USERS* in our case. To satisfy the JPA requirements, the *Name* is made to implement the *Serializable* interface along with a package-specific default constructor. The remaining decorations are self-explanatory.

The following is the decorated *PhoneNumber* class and there is nothing special about it. This is also an immutable *ValueObject* with appropriate JPA and validation annotations.

```

@Embeddable
public class PhoneNumber {
    @Column(name = "phone")
    @Min(90000000000L)
    private long value;
    PhoneNumber() {}
    ...
}

```

Finally, we come to the *UserRepository*. Spring Boot generates repositories with standard capabilities to save, find and delete the objects based on mere interface definition. You need not really implement this interface. Here is the decorated *UserRepository*:

```

@Repository
public interface UserRepository extends JpaRepository<User,
Name> {
}

```

We are done with implementing the domain layer of *AddService*.

Application layer

As you know, the application layer acts as the intermediary between the domain layers and the external clients, like REST clients in our case. It is responsible for receiving data transfer objects (DTOs) from the client and delegating the work to the appropriate domain objects. It is also responsible for doing the opposite while rendering the response.

The application layer of *AddService* consists of *UserController* along with a couple of DTOs. The DTOs do not require any sort of decoration.

Here is the decorated *UserController* with implementation:

```

@RestController
public class UserController {
    @Autowired
    private UserRepository repo;
    @Autowired
    private Validator validator;
    @PostMapping("/user")
    public ResponseEntity<UserRecord> add(@RequestBody
NewUser newUser) {
        Name name = new Name(newUser.name);
        PhoneNumber phoneNumber = new PhoneNumber(newUser.
phone);
        User user = new User(name, phoneNumber);
        Set<ConstraintViolation<User>> violations = validator.
validate(user);
        ResponseEntity<UserRecord> response;
        if (!violations.isEmpty())
            response = new ResponseEntity<>(HttpStatus.BAD_
REQUEST);
        else {
            repo.save(user);
            UserRecord record = new UserRecord(
                user.getName().getValue(),
                user.getPhone().getValue(),
                user.getSince());
            response = new ResponseEntity<>(record,
HttpStatus.OK);
        }
        return response;
    }
}

```

The annotation `@RestController` makes the *UserController* capable of handling the HTTP and JSON out-of-the-box. This is injected with *UserRepository* and *Validator* for the obvious purpose.

The annotation `@PostMapping("/user")` maps the POST REST API to the *add()* method. The Spring Boot takes care of marshalling the HTTP Request POST body to the *NewUser* DTO because of the `@RequestBody` annotation. The implementation of the *add()* method is self-explanatory.

Note that we skipped the messaging feature in the controller. We will refactor the controller to integrate it with a message broker like Kafka in the future.

Bootstrapping

And, finally, the *AddService* acts as the bootstrap for the whole microservice. As already discussed, the Spring Boot microservices are modelled as self-sufficient Java applications with a *main()* method that initialises everything, including an embedded Web server. The `@SpringBootApplication` annotations are just enough to get all the required bootstrap.